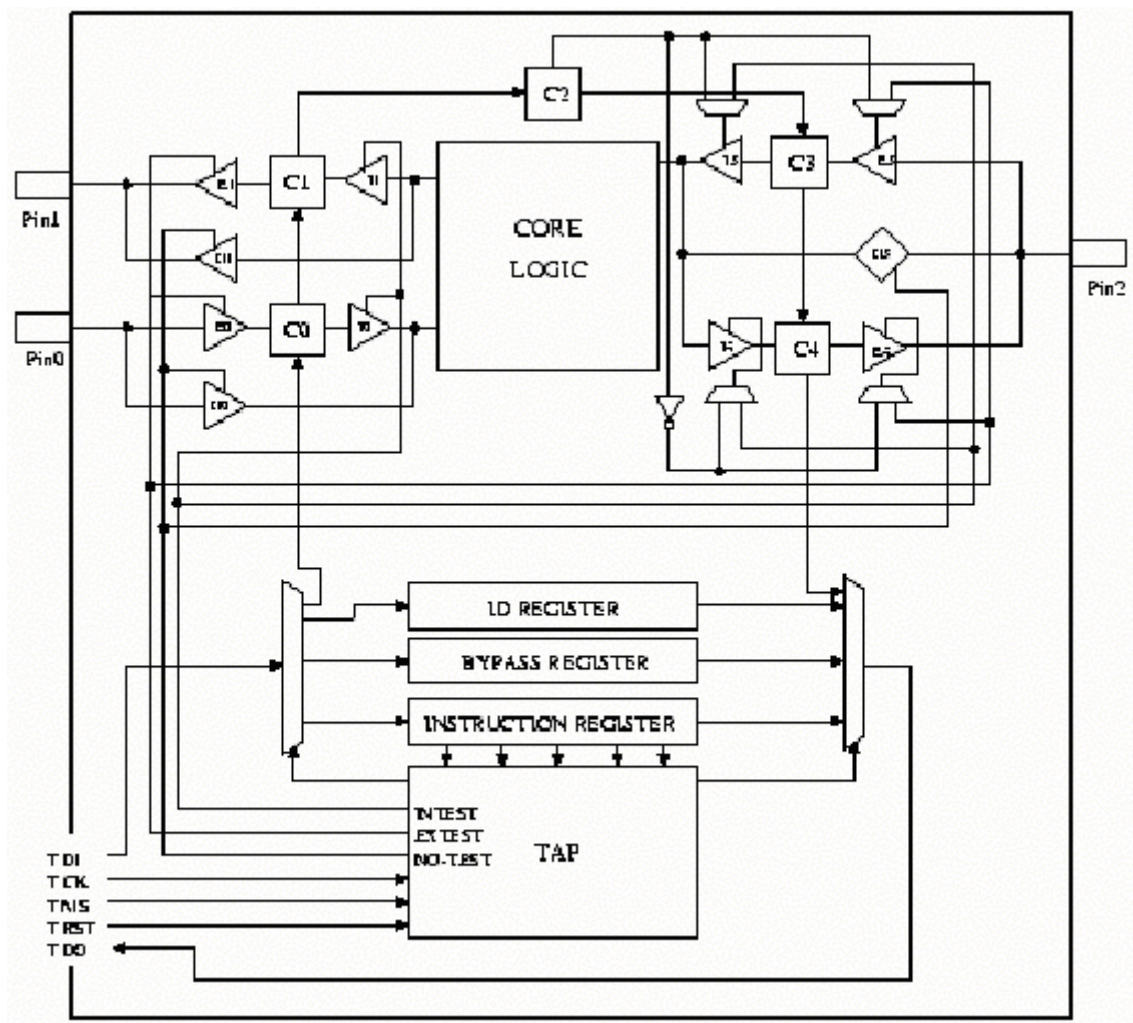


JTAG Boundary Scan Interface

JTAG TAP เป็นมาตรฐานของ IEEE 1149.1 ซึ่งกล่าวอ้างถึง การทดสอบการทำงานของพอร์ตและสถาปัตยกรรม Boundary – Scan



การทำงาน

สัญญาณทั้งหมดที่อยู่ระหว่าง core logic และเส้นทางเดินของสัญญาณขา ของตัวชิพซึ่งเรียกว่า “Boundary Scan Register” (BSR) จากรูปแสดง ด้วยเซลล์ C1 , C2 , C3 , C4 และมีเส้นทางต่อจากขาไปยัง core logic ในการทำงานในโหมด external-test mode ขาจะไม่สามารถต่อถึงเข้ากับ core logic ได้ จากรูปมี ขา Pin1 กับขา

Pin2 เป็นเอาต์พุต และจะทำการอ่านค่าแล้วทำการ latch โดยขาอินพุตคือ ขา Pin0 กับขา Pin2 แต่ถ้าเป็นการทำงานใน internal-test mode ขาจะไม่ต่อถึงกับ core logic แต่ core logic จะเป็นตัวรับสัญญาณอินพุต และจะทำการอ่านค่าและ latch สัญญาณเอาต์พุตที่ core logic

จากรูป สมมติให้ JTAG ทำงานในโหมด external-test mode โดย C0 เป็น BSR เซลล์ แล้วทำการอ่านค่าที่ขาอินพุต Pin0 , C1 เป็น BSR เซลล์เอาต์พุตที่ขา Pin1 , C2 ไม่มีการตอบสนองการทำงานกับขาใดๆ แต่มันจะ “enable” BSR เซลล์ ซึ่งควบคุมการทำงานแบบทางเดียว จากการทำงานที่เป็นสองทางที่ขา Pin2 , C3 เป็นอินพุต BSR แบบสองทางที่ต่อกับขา Pin2 และ C4 เป็นเอาต์พุต BSR เซลล์ แบบสองทางที่ต่อกับขา Pin2 เราสามารถสรุปการทำงานของ BSR เซลล์ได้ 3 อย่าง ดังนี้

- Input Cells คือ C0 , C3 ซึ่งจะทำงานสัมพันธ์กับขาที่ต่ออยู่ในการ Capture เมื่อต่อกับ JTAG ขณะทำงานใน external test mode
- Output Cells คือ C1 , C4 ซึ่งจะทำงานสัมพันธ์กับขาที่ต่ออยู่ในการให้ค่าออกมา ขณะทำงานใน external test mode
- Enable Cells คือ C2 จะไม่ขึ้นอยู่กัขาใดๆ แต่จะควบคุมให้มีการไหลของข้อมูลเพียงทางเดียวหรือว่าสองทาง หรือจะกำหนดให้ขาสัญญาณเป็นอินพุตหรือเอาต์พุต

ส่วนเกท E0 , E3 และ E4 การทำงานจะถูกควบคุมโดย TAP (และอาจควบคุมด้วยการ “enable” เซลล์ เหมือนกับ C2) การอ่านค่าหรือนำค่าไปใช้ จะเป็นไปตามลำดับของสถานะ(State)อินพุตหรือเอาต์พุต , เซลล์ , จาก ขาของตัวชิพ สถานะ(State)ในการอ่านค่า (capture) หรือ การนำค่าไปใช้ (apply) ซึ่งจะมีช่วงเปลี่ยนการทำงานที่แน่นอนในตาม TAP state-machine ซึ่งอาจจะเป็นตอนที่ IR (instruction register) มีการโหลดค่าก่อนแล้วนำมาเก็บ เป็น opcode (EXTEST)

เกท I0,I3และ I4 การทำงานจะถูกควบคุมโดย TAP (และอาจควบคุมด้วยการ “enable” เซลล์ เหมือนกับ C2) การอ่านค่าหรือนำค่าไปใช้ จะเป็นไปตามลำดับของสถานะ(State)อินพุตหรือเอาต์พุต , เซลล์ , จาก สัญญาณภายในลอจิกของตัวชิพสถานะ(state)ในการอ่านค่า(capture) หรือ การนำค่าไปใช้ (apply) ซึ่งจะมีช่วงเปลี่ยนการทำงานที่แน่นอนใน TAP state-machine state-machine ซึ่งอาจจะเป็นตอนที่ IR (instruction register) มีการโหลดค่าก่อน แล้วนำมาเก็บ เป็น opcode (INTEST).

เกท N0 , N1 และ N3 จะทำงานเมื่อระบบอยู่โหมดปกติ (normal-operation mode) เท่านั้น และขาของชิปต่อถึงกับ core-logic ค่ารีจิสเตอร์ของ BSR สามารถเขียนและอ่านได้บิตต่อบิต ตามลำดับ โดยใช้สัญญาณ TDI และ TDO ของ JTAG ในความเป็นจริงในการอ่านหรือเขียน BSR ในช่วงเวลาเดียวกันของค่าใหม่จากขาสัญญาณ TDI ขณะที่มีการ shift ค่าออกจากขาสัญญาณ TDO จะใช้เทคนิคเดียวกันในการอ่านและเขียนค่าของ JTAG registers ตัวอื่นๆ ด้วย โดยจะใช้ TAP controller เป็นตัวต่อระหว่างขา TDI , TDO กับตัว BSR

สัญญาณที่ใช้ในการติดต่อ

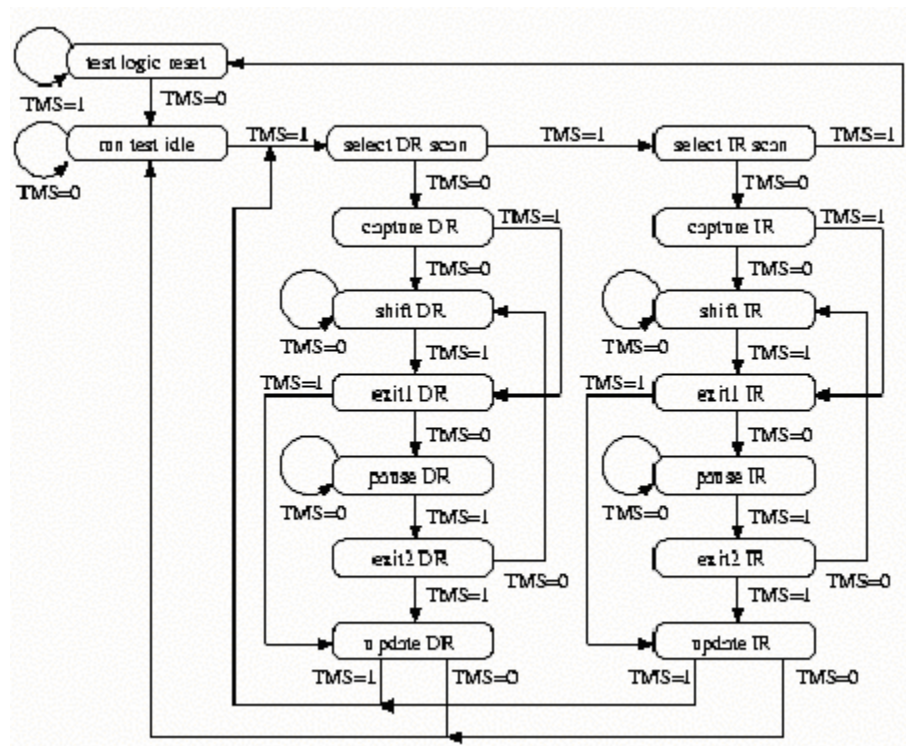
JTAG ประกอบด้วยขาสัญญาณที่ใช้สำหรับควบคุมในการทำงาน 5 ขา คือ

- TRSC - Test Reset Input (active low) เป็นสัญญาณอะซิงโครนัสของ TAP controller ที่ใช้ในช่วงเริ่มต้นการติดต่อและยกเลิกการติดต่อ
- TCK - Test Clock Input เป็นสัญญาณคล็อกจากภายนอกไม่ได้ขึ้นกับระบบ
- TMS - Test Mode Select Input ใช้ควบคุมการเปลี่ยนโหมดการทำงานของ state machine
- TDI - Test Data Input เป็นขาที่ใช้ชีพข้อมูลไปยัง JTAG register หรือ Boundary Scan chain (Boundary Scan Register, หรือ other data registers อื่นๆ) โดยข้อมูลจะผ่านทางขา TDI
- TDO - Test Data Output เป็นขาที่ใช้ชีพข้อมูลออกจาก JTAG register หรือ Boundary Scan chain

โดยปกติ การทดสอบการทำงานของบอร์ดที่ประกอบขึ้นด้วยจำนวนชิพหลายตัว ที่สนับสนุนการทำงานแบบ JTAG จะต่อสัญญาณ TRST, TCK และ TMS แบบขนานกับชิพทุกตัว และต่อขาสัญญาณ TDO จากชิพตัวหนึ่งไปยัง TDI อีกตัวในลักษณะลูป

TAP controller

เป็นตัวที่ใช้สำหรับควบคุมการติดต่อขาสัญญาณของ JTAG กับ BSR ของชิพ ซึ่งมีขั้นตอนการทำงานตาม state-machine ดังรูปข้างล่าง โดยการเปลี่ยนของสถานะ(State) จะขึ้นกับขาสัญญาณ TMS



รูปที่ 15 ภาพแสดงไดอะแกรมของการเปลี่ยนสถานะ

จากภาพไดอะแกรมข้างบน จะเห็นว่าการออกจากสถานะ(State)มีสองทาง การเปลี่ยนของสถานะ(State) จะขึ้นอยู่กับสัญญาณ TMS เพียงสัญญาณเดียว ในไดอะแกรมจะมีสองทางหลักในการเปลี่ยนการทำงานของ (State)ซึ่งเป็นการควบคุมการทำงานของ Data Register (ID register , Bypass register , BSR register) และ Instruction register โดย Data Register จะมีการทำงานทุกครั้งเมื่อทางเดินของ Data Register ถูกเลือกด้วยการโหลดค่าลงใน Instruction Register

การเปลี่ยนเส้นทางตามลำดับของโปรแกรมข้างล่าง (โดยปกติเรียกว่า IR path) เป็นการโหลดค่าใหม่ใน Instruction Register และ อ่านค่าเดิมออก

```
*-> test logic reset
--> run test idle
--> select dr scan
--> select ir scan
--> capture ir
--> shift ir --> ... n times ... --> shift ir
--> exit1 ir
--> update ir
--> run test idle ->*
```

ค่าใหม่จะถูก shift ลงใน Instruction Register จากขาสัญญาณ TDI เป็นจำนวน 1 bit ซึ่งขึ้นอยู่กับสถานะ "shift ir" ทั้งหมด ส่วนค่าเก่าของ Instruction Register ถูก shift ออกจากขาสัญญาณ TDO เป็นจำนวน 1 บิต ซึ่งเป็นการออกจากสถานะ "shift ir"

การเปลี่ยนเส้นทางตามลำดับของโปรแกรมข้างล่าง (โดยปกติเรียกว่า DR path) เป็นการโหลดค่าใหม่ใน Data Register ที่ได้เลือกไว้ในขณะนั้นแล้ว อ่านค่าเดิมออกไป

```
*-> test logic reset
--> run test idle
--> select dr scan
--> capture dr
--> shift dr --> ... n times ... --> shift dr
--> exit1 dr
--> update dr
--> run test idle ->*
```

ค่าใหม่ถูก shift ลงใน Data Register ที่เลือกไว้ในขณะนั้น จากขาสัญญาณ TDI เป็นจำนวนหนึ่งบิต ซึ่งการทำงานขึ้นอยู่กับสถานะ "shift dr" ทั้งหมด ส่วนค่าเก่าจะออกจาก Data Register ที่เลือกไว้ในขณะนั้นจะถูก shift ออกจากขาสัญญาณ TDO เป็นจำนวนหนึ่งบิต ซึ่งเป็นการออกจากสถานะ "shift dr" และค่าของ Instruction Register จะถูก shift ออกไป การ captured ขึ้นอยู่กับสถานะ "capture ir" ค่าของ Data Register ที่เลือกไว้ในขณะนั้น จะถูก shift ออกไป การ captured ขึ้นอยู่กับสถานะ "capture dr" ค่าใหม่จะถูก shift ลงใน Instruction Register ซึ่งเป็นการนำค่ามาใช้ (และคำสั่งจะเป็นผล) ซึ่งขึ้นอยู่กับสถานะ "update ir" ค่าของ Data Register ที่เลือกไว้ในขณะนั้นจะถูก นำไปใช้ (คือ การส่งข้อมูลออกที่ขาเอาต์พุต ในกรณีของ BSR) ซึ่งขึ้นอยู่กับสถานะ "update dr"

Data register

เมื่อมีการพูดถึงวิธีการทดสอบการทำงาน จะเป็นผลของการเก็บค่าของ instruction register ซึ่งเป็นการเลือกกระหว่างความแตกต่าง ของ data registers ที่ทำงานในช่วง "dr path" ซึ่งต่อไปนี้เป็นารแสดง data-registers ที่ใช้ใน JTAG

- *Device ID register (IDR)* reads-out เป็นการกำหนดหมายเลขให้กับชิปที่นำมาเชื่อมต่อ
- *Bypass register (BR)* 1-cell pass-through register ซึ่งเป็นการต่อ TDI ด้วย TDO กับหน่วงเวลาไป 1clock เพื่อให้ง่ายในการติดต่อกับอุปกรณ์อื่นที่อยู่บนแผ่น PCB เดียวกัน
- *Boundary Scan register (BSR)* ซึ่งอธิบายรายละเอียดเกี่ยวกับสัญญาณที่อยู่ระหว่าง core-logic กับขา

Instructions

โดยปกติของการทดสอบด้วย JTAG เป็นการเข้าสู่คำสั่งซึ่ง เป็นลักษณะชนิดของการทดสอบที่จะกระทำต่อไป และ Data Register ถูกใช้ในช่วงที่มีการทดสอบ ค่าภายใน Instruction Register (จากวิธีการทำงานของ TAP ตลอดถึงการ ใช้ "ir path") และการใช้ Data Register ในการทดสอบ (จากวิธีการทำงานของ TAP ตลอดถึงการ ใช้ "dr path" เพียงหนึ่งตัวหรือมากกว่า)

มีคำสั่งที่เป็นคำสั่งเฉพาะ (Private instructions) และ คำสั่งทั่วไป (Public instructions) โดยคำสั่งทั่วไปได้มาจากเอกสารของโรงงานผู้ผลิตชิพ และใช้กันโดยทั่วไป ส่วนคำสั่งเฉพาะจะไม่เปิดเผย ซึ่งมาตรฐานของ IEEE – 1149 เป็นการกำหนดการใช้ชุดคำสั่งทั่วไปที่แสดงใน JTAG ทั้งหมด โดยข้างล่างนี้เป็นชุดคำสั่งดังกล่าว

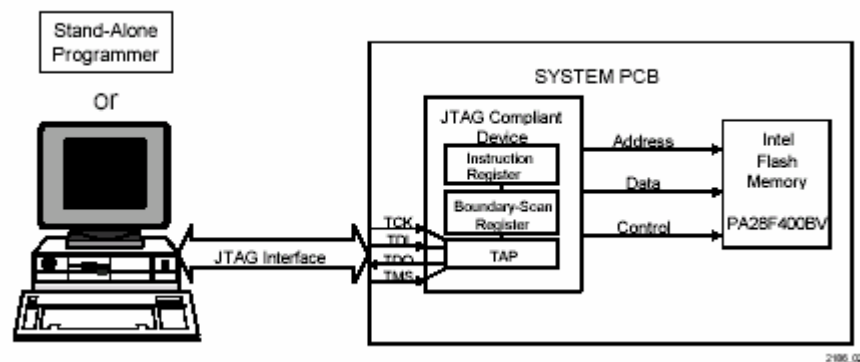
- *BYPASS* : นี้เป็นขาสัญญาณ TDI และ TDO ที่ต่อผ่านเข้าไปในรีจิสเตอร์เพียง 1 บิต (single-bit pass-through register) นี้เป็นคำสั่งที่ยอมให้ต่อเข้ากับอุปกรณ์ตัวอื่นในลักษณะที่เหมือนกับบูป
- *EXTEST* : นี้เป็นคำสั่งเกี่ยวกับ boundary scan register (BSR) ที่ต่อระหว่าง สัญญาณ TDI และ TDO มีการ capture ค่าที่ขาของชิพ โดย BSR เซลล์ ซึ่งทั้งหมดจะเป็นการทำงานในสถานะ "capture dr" (ดูการเปลี่ยน สถานะใน diagram) ค่าที่เก็บใน BSR register จะถูก shift ออกไป ผ่านทางขาสัญญาณ TDO แล้วจะออกจากสถานะ "shift dr" ค่าที่เก็บใน BSR register (ที่ได้จากการ capture) จะถูก shift

ออกไป แล้วค่าใหม่จะถูก shift เข้ามา ซึ่งการทำงานทั้งหมดจะอยู่ในสถานะ "shift dr" ค่าใหม่ที่เก็บใน BSR และถูกนำมาใช้ โดยเป็นสัญญาณที่ขาของชิพ และจะทำงานในช่วงสถานะ "update dr"

- *IDCODE* : โดย ID register ที่ต่อระหว่าง TDI และ TDO ทั้งหมดจะเป็นการทำงานในสถานะ "capture dr" เป็น Device ID Code (เป็นการเชื่อมต่ออุปกรณ์ โดยมีการกำหนดหมายเลขจากโรงงานผู้ผลิต)
- *INTEST* : นี่เป็นคำสั่งเกี่ยวกับ boundary scan register (BSR) ที่ต่อระหว่าง สัญญาณ TDI และ TDO สัญญาณภายใน core-logic ของชิพ จะถูกcapture โดย BSR เซลล์ ซึ่งการทำงานทั้งหมดจะอยู่ในสถานะ "capture dr" (ดูการเปลี่ยน สถานะใน diagram) ค่าที่เก็บใน BSR register จะถูก shifted ออกผ่านทางขาสัญญาณ TDO ซึ่งจะออกจากสถานะ "shift dr" ค่าที่เก็บใน BSR register (ที่ได้จากการ capture) จะถูก shift ออกไป แล้วค่าใหม่จะถูก shift เข้ามา ซึ่งการทำงานทั้งหมดจะอยู่ในสถานะ "shift dr" ค่าใหม่ที่เก็บใน BSR และถูกนำมาใช้ เป็นสัญญาณให้กับ core-logic ในช่วงที่ทำงานในสถานะ "update dr"

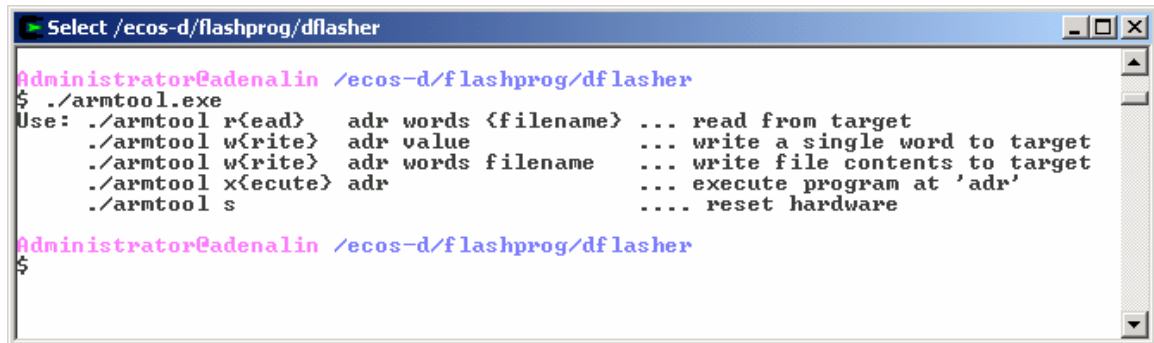
On-Board Programming

สิ่งจำเป็นอันดับแรกก่อนที่จะนำโปรแกรมที่พัฒนาไปทำงานยังที่แพลตฟอร์มที่ได้ออกแบบขึ้นจำเป็นต้องมีโปรแกรมที่มีการติดต่อระหว่างเครื่องโฮสต์(Host)และเครื่องแพลตฟอร์ม(Target) หรือบอร์ดควบคุม(On-Board Programming) ขึ้นตอนนี้ได้ทำการพัฒนาเครื่องมือที่ใช้ในการโปรแกรมหน่วยความจำแฟลช (Flasher) ซึ่งพัฒนาต่อจากโปรแกรม "Armtool" โดยจะส่งข้อมูลผ่าน พอร์ตขนานทำการโปรแกรมผ่านพอร์ต JTAG ของไมโครคอนโทรลเลอร์ ดังรูปข้างล่างแสดงการทำงานของ On-Board Programming



รูปที่ 15 การทำ On-Board Pramming

โปรแกรม “Armtool” จะเป็นมีอินเตอร์เฟสเป็นคอมมานด์ไลน์ทำงานได้ทั้งในระบบปฏิบัติการวินโดวส์ Windows + Cygwin(Cygwin= เป็นโปรแกรมที่จำลองสภาพแวดล้อมของระบบปฏิบัติการยูนิกซ์บนวินโดวส์) และระบบปฏิบัติการลินุกซ์ ดังรูป



```
Select /ecos-d/flashprog/dfletcher
Administrator@adenalin /ecos-d/flashprog/dfletcher
$ ./armtool.exe
Use: ./armtool r{read}  adr words {filename}  ... read from target
      ./armtool w{write} adr value           ... write a single word to target
      ./armtool w{write} adr words filename   ... write file contents to target
      ./armtool x{execute} adr                ... execute program at 'adr'
      ./armtool s                        .... reset hardware
Administrator@adenalin /ecos-d/flashprog/dfletcher
$
```

รูปที่ 16 อินเตอร์เฟสของโปรแกรม Armtool บนวินโดวส์

โปรแกรม “Armtool” สามารถที่จะทำงานเป็นตัวโหลด หรือ อ่านข้อมูลของหน่วยความจำ ยกเว้น หน่วยความจำแฟลช ซึ่งต้องเขียนโปรแกรมเพิ่มขึ้นมา คำสั่งที่เป็นประโยชน์อีกคำสั่งคือ คำสั่ง เอกซคิว เพื่อเป็นการสั่งให้ ARM7TDMI ทำงานตามที่ระบุในแอดเดรส สำหรับการโปรแกรมหน่วยความจำแฟลชจะใช้โปรแกรม Flasher ซึ่งอาศัยการทำงานของ Armtool และ Flasher code ที่เขียนขึ้น

การใช้งานโปรแกรม armtool

โปรแกรม armtool มีการใช้งานอยู่ 5 รูปแบบด้วยกันดังรูปที่ 16

1. การอ่านข้อมูลต้องระบุ OPTION read แล้วตามด้วย address ที่ต้องการอ่าน จำนวน word (4 เท่าของ byte) และ filename ที่ต้องการ save
2. การเขียนข้อมูลสามารถระบุได้ 2 แบบ ต้องระบุ OPTION write ตามด้วย address ส่วน file ที่เหลือก็ใช้งานเช่นเดียวกับ read
3. การ Execute Program ต้องระบุ OPTION x แล้วตามด้วย address ที่ต้องการให้ run program
4. Hardware reset ทำได้โดยการระบุ OPTION s

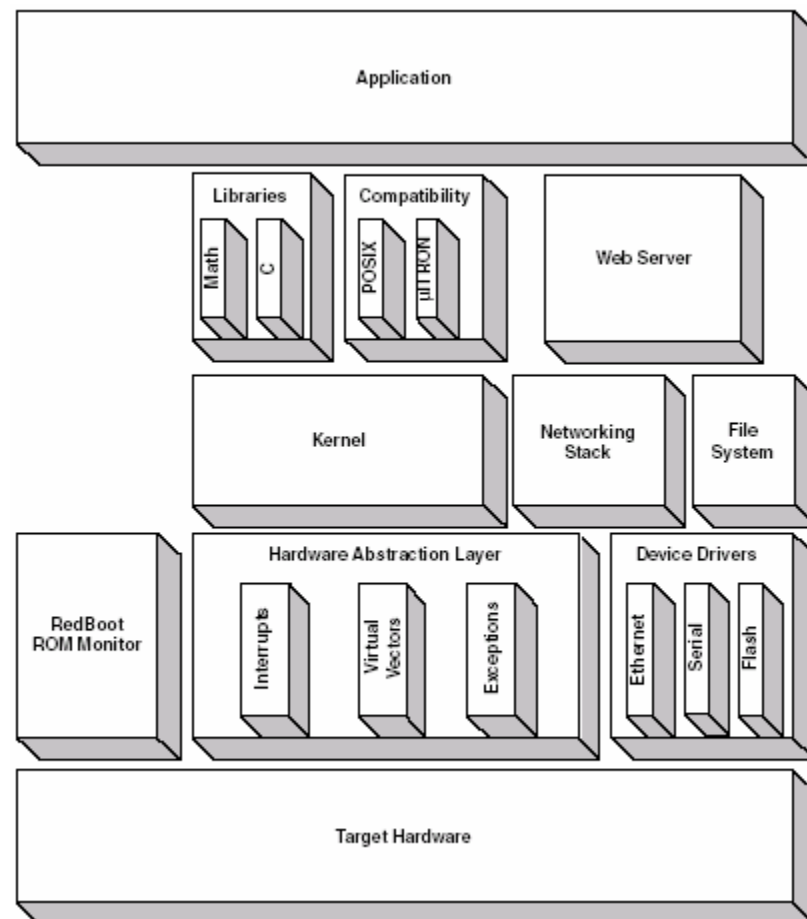
eCos (Embedded Configurable Operating System)

eCos เป็นระบบปฏิบัติการพัฒนาเป็นครั้งแรกในปี 1997 เป้าคือเพื่อพัฒนาระบบฝังตัวประสิทธิภาพสูง ต้องการทรัพยากรที่น้อยลง นอกจากนี้ยังเป็นระบบปฏิบัติการที่มีการเผยแพร่แจกจ่ายซอร์สโค้ดฟรี ถูกออกแบบให้สามารถรองรับแอปพลิเคชันต่างๆ ตามที่ผู้พัฒนาต้องการต้องการไม่ว่าจะเป็นด้านการติดต่อสื่อสาร การควบคุมเครื่องจักร และแอปพลิเคชันอื่นๆ ในระดับสูง

eCos เป็นระบบปฏิบัติการที่เป็นเรียลไทม์ (Real – Time Operathing System)มีความเหมาะสมกับระบบฝังตัวระดับลึกที่ระบบปฏิบัติการลินุกซ์ทั่วไปไม่สามารถทำได้ โดยทั่วไประบบปฏิบัติการลินุกซ์ จะมีขนาดของเคอร์เนล อยู่ที่ประมาณ 500 กิโลไบต์ ถึง 1.5 เมกะไบต์ แต่ eCos จะมีขนาดของเคอร์เนลประมาณ 10 - 100 กิโลไบต์ และอาจขึ้นอยู่กับความต้องการของระบบด้วย

สถาปัตยกรรม

eCos ถูกออกแบบมาให้สนับสนุนกับแอปพลิเคชันที่มีการทำงานแบบเรียลไทม์(real-time) เหมาะที่จะนำมาพัฒนาแอปพลิเคชันของระบบฝังตัว ที่ระบบปฏิบัติการลินุกซ์ทั่วไปไม่สามารถทำได้ โดยทั่วไประบบปฏิบัติการลินุกซ์ จะมีขนาดของเคอร์เนล อยู่ที่ประมาณ 500 กิโลไบต์ ถึง 1.5 เมกะไบต์ แต่ eCos จะมีขนาดของเคอร์เนลประมาณ 10 - 100 กิโลไบต์ และอาจขึ้นอยู่กับความต้องการของระบบด้วย รองรับกับไดรเวอร์ของอุปกรณ์ ,ไฟล์ระบบ ,โปรโตคอล TCP/IP การจัดการหน่วยความจำ , exception handling , ภาษาซีและไลบรารี ,และอื่น ๆ eCos มีเครื่องมือที่ใช้ในการ



รูปที่ 17 สถาปัตยกรรมของระบบปฏิบัติการ eCos

พัฒนาแอปพลิเคชันระบบฝังตัว โดยได้รับการสนับสนุนตัวคอมไพเลอร์ , แอสเซมเบลอร์ , ลิงเกอร์ , ดีบั๊กเกอร์ และซิมูเลเตอร์ จาก กนู(GNU) ดังนี้

- สนับสนุนไครว์เวอร์อินเทอร์เน็ต USB , flash , serial และอื่นๆ
- สนับสนุนการทำงาน TCP/IP แบบ SNMP
- สนับสนุนภาษาซีและไลบรารี
- รองรับมาตรฐาน POSIX EL/XI ระดับ 1
- สนับสนุน μ ITRON 3.02
- ได้รับการสนับสนุนเครื่องมือจากกนู(GNU)
- สนับสนุนดีบั๊กมอนิเตอร์ RedBoot
- มี Hardware Abstraction Layer (HAL) ทำให้สามารถทำงานได้กับโปรเซสเซอร์หลายแพลตฟอร์ม
- ส่วนประกอบของเคอร์เนลที่เป็นเรียลไทม์ :
 - Interrupt handling
 - Exception handling
 - Choice of schedulers
 - Thread support
 - Rich set of synchronization primitives
 - Timers, counters, and alarms
 - Choice of memory allocators
 - Debug and instrumentation support

สนับสนุนระบบปฏิบัติการโฮสต์

- Red Hat Linux
- Sun Solaris and ระบบ UNIX อื่นๆ
- Microsoft Windows

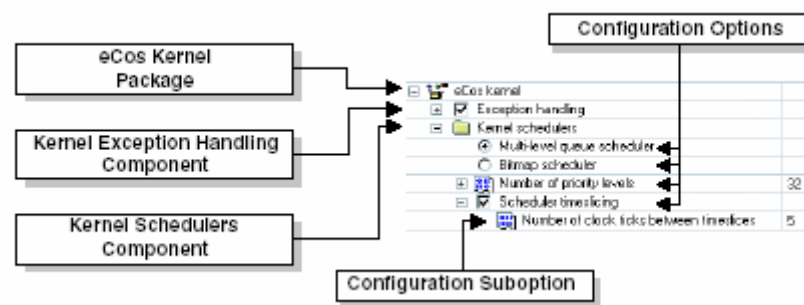
แพลตฟอร์มที่สนับสนุน

- ARM
- Hitachi SH3, SH4
- MIPS
- NEC V850
- Panasonic AM3x

- PowerPC
- SPARC
- x86
- SuperH, Matsushita AM3x, IA32, และอื่นๆ

วิธีการของ eCos Configuration

ในระบบฝังตัวต้องมีขนาดเล็ก เร็ว ถูก สามารถทำได้โดยการควบคุมให้มีซอฟต์แวร์ที่จำเป็นอยู่ในระบบเท่านั้น ซึ่งจะแตกต่างจากวิธีการที่กำหนดให้มีคอนโพเนนท์เข้าไปรวมกับแอปพลิเคชัน



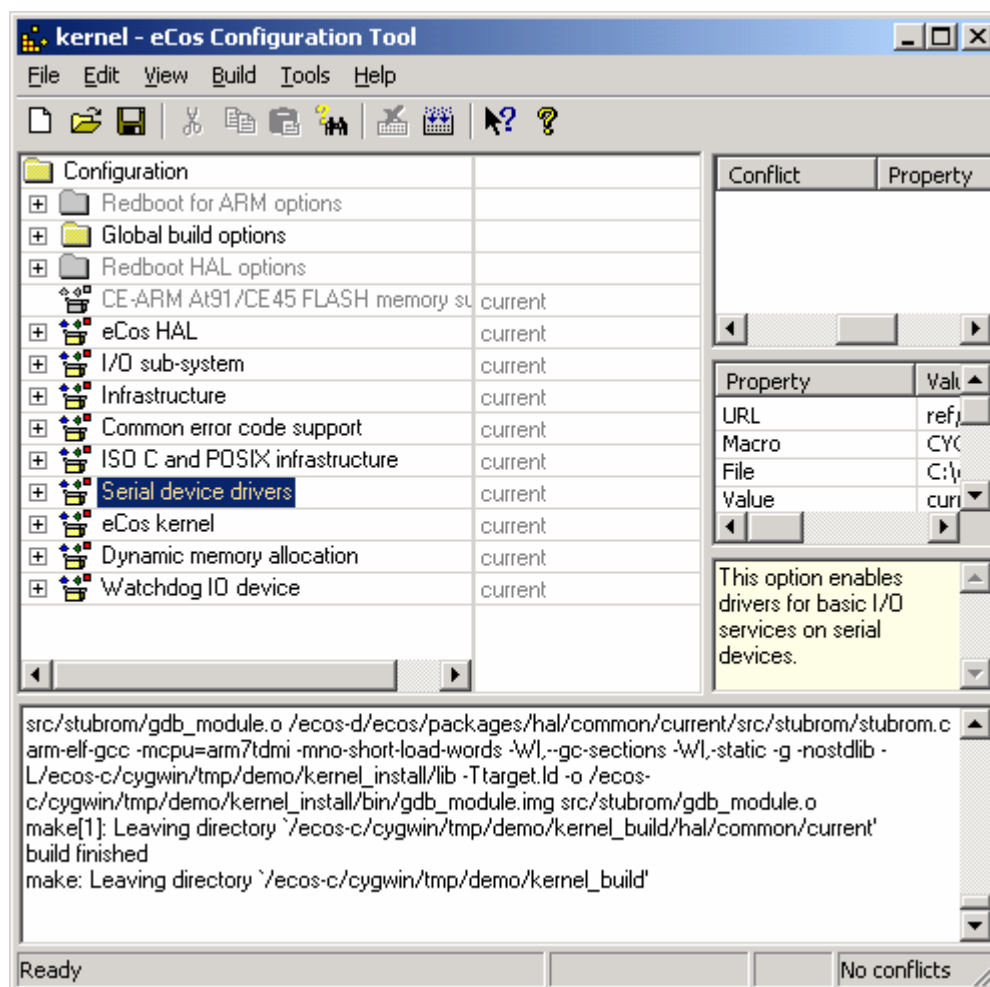
รูปที่ 18 การ configuration eCos

การลดขนาดแอปพลิเคชันระบบฝังตัว eCos จะใช้งาน Compile-time control โดยให้นักพัฒนาระบบจัดการกับคอมโพเนนท์ และเรียกใช้คอมโพเนนท์นั้นๆ สำหรับสร้างแอปพลิเคชันตามที่ตั้งใจ ซึ่งจะช่วยให้ลดขนาดของโค้ดได้ดีที่สุด เพราะว่าเป็นการควบคุมที่ระดับ Statement ใน source code มากกว่าที่จะไปควบคุมที่ Function หรือ object การใช้งาน Compile-time จึงเหมาะสำหรับการพัฒนาระบบฝังตัว นอกจากโค้ดจะมีขนาดเล็กแล้วยังมีข้อดีที่สำคัญคือ

- แอปพลิเคชันทำงานได้เร็วกว่าเพราะว่าไม่มีการตรวจสอบตัวแปรในระหว่าง run-time เพื่อหาว่าต้องทำอะไร
- โปรแกรมมีการตอบสนองมากกว่า และ ส่วนที่ซ้ำซ้อนจะถูก reused ซึ่งเป้าหมายในการสร้างระบบตรวจสอบสำหรับ real-time device
- โค้ดที่ง่ายกว่าจะถูกสร้างขึ้น ทำให้ตรวจสอบ และ ทดสอบทำได้ง่ายขึ้น
- ทำให้ลดค่าใช้จ่ายเพราะว่าทรัพยากรที่ใช้จะถูก optimize และการใช้งาน processor cycle อย่างมีประสิทธิภาพ ด้วยเหตุนี้จึงต้องการ hardware ที่มีราคาถูกกว่าในการออกแบบ

eCos Configuration tool

eCos Configuration tool เป็นเครื่องมือที่ใช้ในการพัฒนาระบบปฏิบัติการอีคอส(eCos) ซึ่งเป็นตัวที่ใช้ในการปรับแต่งแก้ไขซอร์สโค้ด ก่อนที่จะทำการรวบรวมเป็นส่วนประกอบหรือคอมโพเนนต์ของไฟล์ตามที่ต้องการ โดยซอร์สโค้ดดังกล่าวจะเก็บรวบรวมไว้ในคอมโพเนนต์ที่ชื่อว่า “*component repository*” คอมไฟล์เพื่อเก็บเป็นไลบรารี สำหรับนำไปลิงค์รวมกับ โปรแกรมแอปพลิเคชันที่เขียนขึ้น



รูปที่ 19 eCos Configuration tool

วิธีการที่จะนำโปรแกรมลงไปทำงานบนบอร์ดทำได้ 2 วิธี ดังนี้

1. การ Burning คือ การนำเอาโปรแกรมและ Rom Filesystem image ที่ผ่านการคอมไพล์เรียบร้อยแล้วไปใส่ในหน่วยความจำประเภทคงอยู่ (Nonvolatile Memory) เช่น หน่วยความจำแฟลช (Flash Memory)

2. การ Downloading คือการนำเอาโปรแกรมและ Rom Filesystem image ที่ผ่านการคอมไพล์เรียบร้อยแล้วโหลดผ่านทางสายสื่อสารไปใส่ในหน่วยความจำ เช่น พอร์ตขนาน หรือ พอร์ตอนุกรม เป็นต้น โดยการเรียกให้ทำงานให้เริ่มต้นทำงานในตำแหน่งที่ได้โหลดลงไป

ในวิธีแรกนั้นเหมาะสมกับระบบที่ได้มีการพัฒนาเรียบร้อยแล้วและพร้อมที่จะนำไปใช้งาน ดังนั้นจึงได้เลือกใช้วิธีที่สองแทน เพราะในขั้นตอนของการทดลองและทดสอบการทำงานจะต้องมีการแก้ไขอยู่เป็นประจำ รวมทั้งต้องมีการดีบั๊กข้อผิดพลาดต่างๆ อยู่เสมอ ถ้ามีข้อมูลอยู่ในแรมอยู่แล้วจะทำให้สามารถที่จะดึงเอาข้อมูลที่อยู่ในแรมออกมาดูได้อย่างสะดวกยิ่งขึ้น ส่วนการโหลดอิมเมจลงไปทำงานบนบอร์ดนั้น ได้เลือกใช้การโหลดผ่านทางพอร์ตขนานของเครื่องพีซี ซึ่งเชื่อมต่อกับพอร์ต JTAG ของบอร์ด เพราะได้มีการพัฒนาทั้งในส่วนของฮาร์ดแวร์ และซอฟต์แวร์ ที่ใช้การติดต่อกับ JTAG เรียบร้อยอยู่แล้ว

การพัฒนา eCos สำหรับบอร์ดควบคุม

สิ่งที่ต้องทำเพื่อให้ eCos สามารถทำงานได้กับบอร์ดควบคุมที่ออกแบบขึ้นโดยการปรับปรุงในส่วนของ HAL(Hardware Abstraction Layer) โดยการเพิ่มแพลตฟอร์มใหม่เข้าไป ดังนี้

- เพิ่มข้อมูลเกี่ยวกับบอร์ดควบคุมเข้าไปใน Database ให้กับ eCos configuration tool
- เพิ่มรายละเอียดของ Memory Layout
- เพิ่มรายละเอียดของ Memory Controller Initialize
- เพิ่มรายละเอียดของ Interrupt Controller Handling
- เพิ่ม Serial Port device driver สำหรับ GDB และ โปรแกรมสำหรับตรวจสอบ
- เพิ่ม System Timer Initialize และ Control
- เพิ่ม Wallclock driver

หลังจากได้ทำการพัฒนาแพลตฟอร์มใหม่เข้าไป ชื่อ “CE Control board DEMO (CE45)” โดยการปรับปรุงแพลตฟอร์มที่ใกล้เคียงกันคือ ATMEL EB40 ได้ผลเป็นดังรูป (เลือก Template เมนูของ eCos Configuration tool) จากนั้นใช้ eCos Configuration tool ทำการปรับปรุงส่วนของ option ที่ต้องการสำหรับแอปพลิเคชันระบบฝัง ทำการคอมไพล์ซึ่งจะได้เป็นไบนารีเพื่อนำไปลิงก์กับแอปพลิเคชันที่เขียนขึ้น ตัวอย่างของโปรแกรมแอปพลิเคชัน จะถูกเรียกใช้จากฟังก์ชัน `cyg_user_start()`

```
/* File Application.c */
```

```
void cyg_user_start()
```

```
{
```

```

printf("Hello eCos World with CE Control Board DEMO(CE45)");

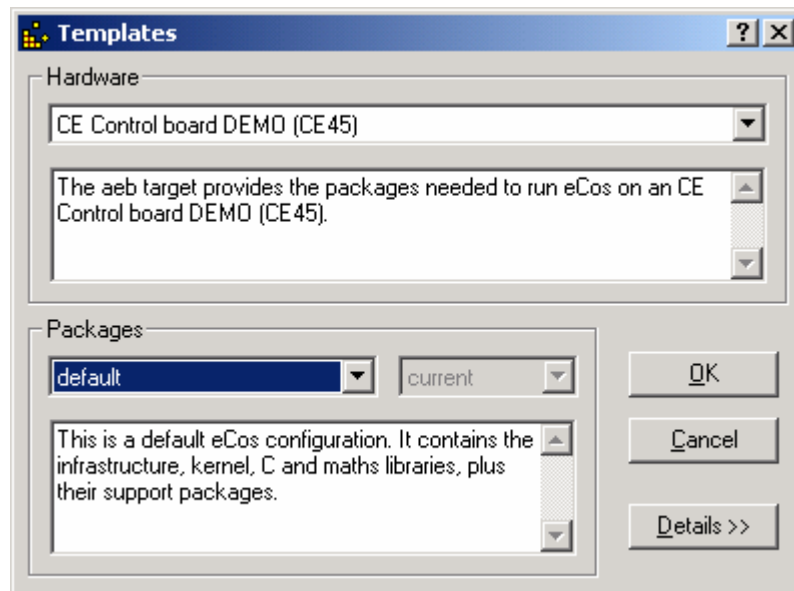
/* To DO โค้ด สำหรับ แอปพลิเคชัน */

}

```

การใช้งาน Platform “CE Control board DEMO (CE45) “

เปิด โปรแกรม configuration tool เลือกเมนู Build->Template ก็จะปรากฏหน้าต่างดังรูป



รูปที่ 20 แสดงแพลตฟอร์ม “CE Control Board DEMO (CE45)”

เลือก Hardware ตามรูป จากนั้นเลือก Packages ที่จะพัฒนา embedded System สำหรับการใช้งาน tool กรุณาอ้างอิงใน ecos user-guides ซึ่งอยู่ใน Directory source/ecos-doc/